LECTURE NOTES

Classification

AI512 — Introduction to Machine Learning

Simon Holm

2026-09-04

SDU

DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1 Binary Classification

Assume that we have a binary classification problem where

$S = \{(x_1, y_1), (x_2, y_2), dots, (x_n, y_n)\}, quad"and"quad y in \{0, 1\}$

Like before we assume the hypothesis to be linear:

$$\mathcal{H} := \{h : h(x) = w^\top x, w \in \mathbb{R}^k\}$$

But in this case the output is now discrete and not continuous.

Let's define one possible way to interpret the label y.

- Sign of h_i
  - If $h_i > 0 \rightarrow$ class 1
  - If $h_i < 0 \rightarrow$ class 0
- magnitude of $|h_i|$
  - Larger $|h_i| \rightarrow$ higher **confidence**
  - Smaller $|h_i| \rightarrow$ lower **confidence**

Note: this is just **one** possible way of interpreting a discrete output.

When interpreted this way, the hypothesis h_i is called the **discriminant function**. Such a model is not easy to train since we normally desire a differentiable loss function, which we can't with this "sign" function. By considering this a probability problem would like to achieve:

$p_i = w^T x_i \quad \text{WRONG!}$

The problem with this is that the range of $w^T x_i \in (-\infty, \infty)$ and $p_i \in [0, 1]$

By taking the log we can widen this so.

$\log(p_i) \propto w^T x_i$

This works well for any $w^T x_i < 0$, but is undefined for $w^T x_i > 0$ since $\log(a) \in (-\infty, 0)$
To fix this, we account for the probability of the other class. We actually want to express

$w^T x_i = \log\left(\frac{\mathbb{P}(y_i=1|x_i)}{\mathbb{P}(y_i=0|x_i)}\right) = \log\left(\frac{p_i}{1-p_i}\right)$

This is called logistic function, and we can solve for p_i to find probabilities for both classes.

$\log\left(\frac{p_i}{1-p_i}\right) = w^T x_i \quad \Rightarrow p_i = \frac{1}{1+e^{-w^T x_i}}$

2 Logistic Regression

We have leaned how binary classification can be done using probability. From that we get that:

$\log\left(\frac{p_i}{1-p_i}\right) = w^T x_i \quad \Rightarrow p_i = \frac{1}{1+e^{-w^T x_i}}$

The function $\sigma(u) = \frac{1}{1+e^{-u}}$ is known as the sigmoid function, and the inverse of the sigmoid function is

$\sigma^{-1}(p) = \frac{p}{1-p} \quad \text{for some } p, \text{given that } u = w^T x_i$

This is called logistic regression.

Now to determine the probability of the whole dataset. We use the following notation

$\mathbb{P}(y_i \mid x_i, w) = p_i^{y_i}(1-p_i)^{1-y_i}$

This way:
- $\mathbb{P}(y_i = 0 \mid x_i, w) = 1 - p_i$
- $\mathbb{P}(y_i = 1 \mid x_i, w) = p_i$

(This is an example of how logistic regression uses probability to determine whether mice are obese or not)

And so to determine the whole dataset we can

$$\prod_{i=1}^{m} p_i^{y_i}(1-p_i)^{1-y_i}$$

Note that products are messy to differentiate, which is nice, for optimization, therefore we can express this as a sum instead:

$$\ell(w) = \log\left(\prod_{i=1}^{m} p_i^{y_i}(1-p_i)^{1-y_i}\right) = \sum_{i=1}^{m}[y_i \cdot log(p_i) + (1-y_i) \cdot log(1-p_i)]$$

Now to express this as a loss function we take the negative log-likelihood, and substitute $p_i = \sigma(w^T x_i)$ (the sigmoid)

$$\mathcal{L}_{01}(w) = -\sum_{i=1}^{m} \log \sigma(w^\top x_i)^{y_i}(1-\sigma(w^\top x_i))^{1-y_i}$$

Now we can differentiate the loss function w.r.t w to find an optimal w. That is stupid, so I won't do it here.

3 Multi-Class Classification

Assume that we now have more than 2 classes (so not binary anymore). Assume that we have C classes, so that

$S = \{(x_1, y_1), (x_2, y_2), dots, (x_n, y_n)\}, quad"and"quad yin\{0, 1, C-1\}$

We will need to model the class probabilities of C different classes: $p_1, p_2, ..., p_C$:

In binary classification, we discriminated classes with a **sign function**. In multi-class classification we ought to assign a **score for each class**:

$s_c = w_c^T x_i, \quad c = 1, 2, ..., C$

Where $s_c \in (-\infty, \infty)$

We want to map these scores to probability. When mapping to probability, we want to make sure that the following constraints hold:

- $0 \leq p_i \leq 1$, each probability is constraint as a number between 1 and 1
- $\sum_{c=1}^{C} p_c = 1$, the sum of all p_c is always 1.

Softmax deals with exactly this, by generalizing the binary classification to $C > 2$

$$\sigma(x)_c = \frac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$$

The related loss function is then:

$$\mathcal{L}_{CE}(W) = -\sum_{i=1}^{m} \log \sigma(w_{y_i}^{\top} x_i) = \sum_{i=1}^{m} \Big\{ -w_{y_i}^{\top} x_i + \log\Big(\sum_{c=1}^{C} e^{w_c^T x_i} \Big) \Big\}$$

where $W = [w_1 ... w_C]$ is a matrix of the weight vectors for each class. This loss function is called the **cross entropy loss**. We will revisit this loss and understand better why it is given this particular name.

Let's try and make an as general algorithm as possible using what we have leaned form regularization

$$L(W) := L_{CE}(W) + \lambda \sum_{c=1}^{C} | \ | \ w_c | \ |_p$$

Like before we can use gradient descent to minimize loss (example from lecture)

4 Performance Metrics for Classification

The goal of classification is to recognize a pattern. We introduce confusion matrices. For confusion matricies:

- TP true and correct class
- FP: false and correct class
- TN true and incorrect class
- FN: false and incorrect class

We can compute many performance metrics from the confusion matrix:

- Accuracy := $\frac{\text{Sum of Diagonal}}{\text{Sum of all samples}}$

- Precision (per class) := $\frac{\text{TP}}{\text{TP+FP}}$
- Recall (per class) := $\frac{\text{TP}}{\text{TP+FN}}$
- F1 Score (per class) := $\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
- False Positive Rate (per class) := $\frac{\text{FP}}{\text{FP+TN}}$
- Specificity := $\frac{\text{TN}}{\text{TN+FP}}$

5 K-Fold Cross Validation

Split up data in k different ways, to optimize

6 K-Nearest Neighbors (kNN) Classifier

kNN is based around memorizing the entire dataset and using distance to neighboring data points to classify new data.

This way a new point is classified based on the classes of its **k nearest neighbors** in the training set.

There are multiple ways to do kNN, here I will show 2 alike, yet different ways.

6.0.1 Standard kNN

$$\hat{y} = \arg\max_{c \in [C]} \sum_{i=1}^{k} \mathbb{1}(y_i = c)$$

The standard kNN selects a class to a new data point, only based on the majority vote of the k-nearest neighbors.

6.0.2 Weighted kNN

$$\hat{y} = \arg\max_{c \in [C]} \sum_{i=1}^{k} \mathbb{1}(y_i = c) \frac{1}{d(x, x_i)}$$

The weighted kNN does the same, however it normalizes each vote, using the ratio of the distance. This way, points with a small distance to the new point x contribute more to the vote, than points with greater distance to the new point.

6.0.3 Voronoi cells

A **voronoi cell** is the area around a datapoint to which it is the closest point Example, any new data point within the blue area, will be classified to the blue

When you do this for all data points, you can create a Voronoi map

7 Receiver Operating Characeteristics (ROC)

ROC curve is a tool to **evaluate the performance of a binary classifier**. Take this example, to find out where the threshold should be (for the model to be optimal), we can use ROC

To understand ROC, we need **two rates**:
1) True Positive Rate (TPR)/ sensitivity / Recall

$$\text{TPR} = \frac{\text{TP}}{\text{TP+FP}}$$

2) False Positive Rate (FPR)

$$\text{FPR} = \frac{\text{FP}}{\text{TN + FN}}$$

W

The ROC plots these as follows for each: The more to better thresholds are the ones most at the top left. Now we can choose one, depending on what the model is trying to achieve.

Also, we can use the **AUC** (area under curve) as measurement for how good the classifier is. The higher the AUC, the better the classifier. A perfect classifier has an AUC of 1. A classifier that performs no better than random guessing has an AUC of 0.5.